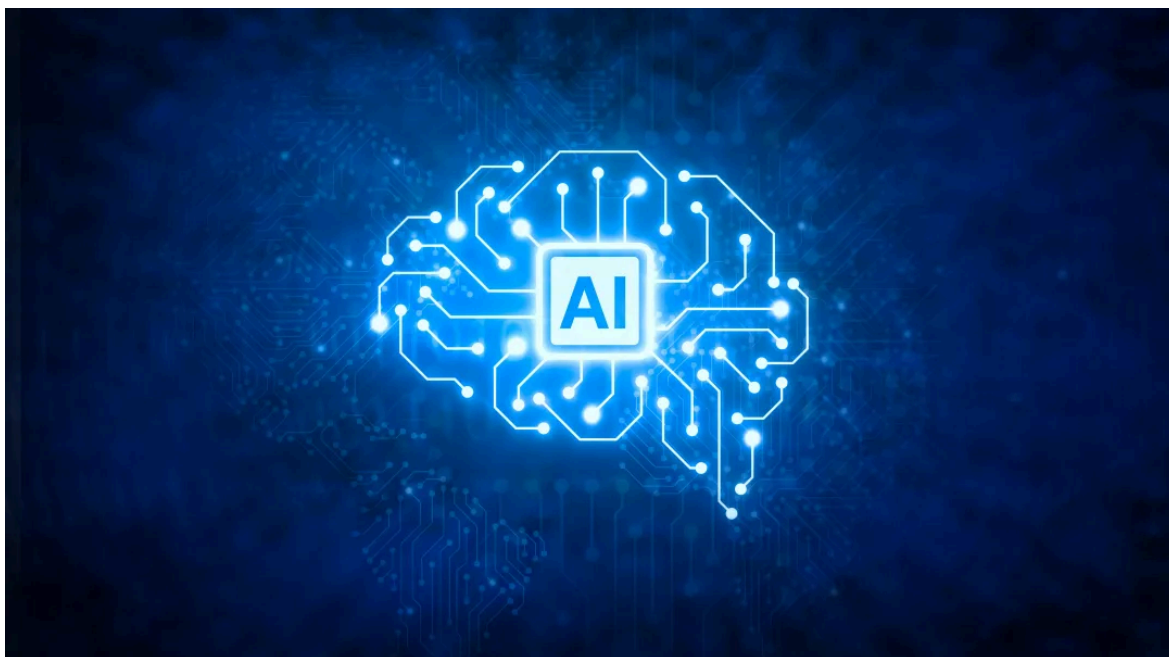


如何更科学、方向可控的实现 Skill 的“自进化”？

Original 飞橇 阿里云开发者 2026年6月9日 08:30 浙江



阿里妹导读



文章内容基于作者个人技术实践与独立思考，旨在分享经验，仅代表个人观点。

背景

现如今，Agent Skills 已经成为 Agent 架构中非常重要的组成部分，是一种知识和工具的资源集合包，Agent 的最终效果与 Skill 的质量有着极高的相关度。关于 Agent Skills 概念的介绍和演化过程，有兴趣的同学可以读我之前的文章《Agent/Skills/Teams架构演进过程及技术选型之道》以及《Agent 核心技术概念与范式发生了哪些演变以及背后的思考》等，我在本文里就不赘述了。

在Agent Skills 的发展最早期的阶段，大家基本上主要就通过人工手写 Skill，这就需要投入大量的精力来进行撰写、调试、优化，效率较低且很难规模化。之后，许多 Agent 的工具或平台都提供了各式各样的 Skill Creator，通过 AI 辅助的方式去帮助用户自动生成 Skill，这在一定程度上提升了 Skill 的生产效率。再后来，就像我前段时间在《深度解析Hermes Agent 如何实现“自进化”及其 Prompt / Context / Harness 的设计实践》中所写的那样，Agent 实现“自进化”的核心方式之一，就是“**Skill 的自动沉淀**”。这个核心逻辑是：每执行完一轮Agent，就会有 Agent 的**运行轨迹（Trajectory）**数据，模型基于这个轨迹的上下文去做判断，是否需要将当前的轨迹总结、沉淀为一个全新的Skill，或者是否需要去更新一个已有的 Skill。这种方式可以说在很大程度上解放了人力，不再需要让人完全从头写 Skill，或者基于已有的 Skill 也不需要做太多手动优化的动作，模型可以自动迭代，从而迈向了一种更加自动化的Skill 构建新形态。

然而，在大家应用Hermes Agent 的过程中，是不是也经常会遇到很多类似如下的问题：

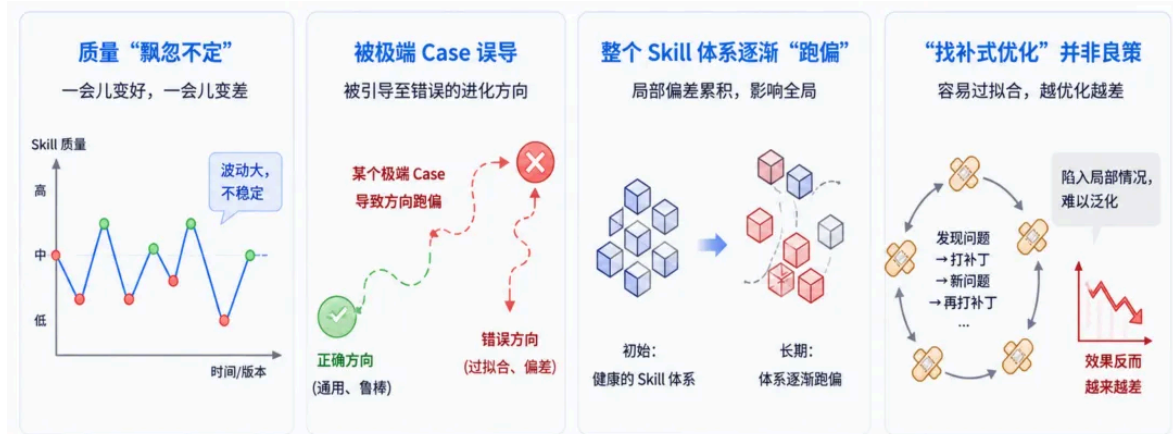
- 为什么自动沉淀的Skill 质量不高？对Agent 的效果帮助不大
- 为什么Skill 更新后，反而比原来的版本效果更差了？
- 原本精简清爽的Skill，经过几次自动迭代后，变的越冗长复杂、难以阅读

那么，今天我们就重点来探讨一下Skill 自进化中遇到的这些坑以及有哪些好的解法。

Skill自进化有哪些难点

使用Hermes 等目前各类 Agent 工具本身提供的方式去做 Skill 的自进化，出现上述这些问题现象，其实也是非常正常的。因为目前的 Skill 自我沉淀机制，大部分都是基于单通Agent 对话轨迹来进行的。这意味着，当前这一轮对话Session 中任务完成的效果，直接决定了这个 Skill 进化的方向。如果这一轮轨迹本身存在偶然性、特殊性，甚至是极端Case，那么基于此生成的Skill 就有可能“带偏”进化的方向。

在个人场景下应用这种Skill 自动沉淀模式，问题其实还不算大。因为个人用户的任务多样性比较高，任务要求相对分散，同一个任务再次运行或遇到完全相同场景的概率相对低一些，Skill 的复用比例不高，因此即使个别 Skill 进化偏差，对整体体验的影响也相对有限，大家可能察觉不到明显的“降智”。



但在企业级场景中，情况就完全不同了。很多企业级Agent 都是每天在线上承接同一类或多类重复任务，会积累垂直领域内海量的、不同用户的Query。虽然它们虽然是同一个任务类型，其运行逻辑、流程应该也是高度趋同的，但由于用户 Query 之间存在差异，因此 Agent 中间产生的结果不同，在层层累积之后，最终执行轨迹和结论也可能会有较大差异，而且还会出现各类边界情况和长尾场景。如果我们简单地根据每一轮的轨迹运行情况，就直接在线上进行Skill 的沉淀或更新，极易导致 Skill 质量“飘忽不定”，稳定性大幅降低。它可能出现“一会儿变好，一会儿变差”的现象，甚至因为某个特定的极端Case 而被引导至错误的进化方向，导致整个 Skill 体系逐渐“跑偏”。这其实与早期的 Prompt 调优也有着很大的相似性。大家可以回顾一下，以前在做提示词调优的时候，如果你只是简单地把badcase 丢给模型，让它根据这些个案去优化提示词，很容易就出现“顾此失彼”的现象。模型为了迎合当前的badcase，甚至会去过拟合（Overfitting），把这些个例情况补进去，不但导致Prompt 变得越来越臃

肿、发散，甚至还会破坏了原本的泛化性和通用性。这种受个例影响而不断“找补式的优化”，并不是一种好的优化方式。所以说，Skill 的自进化如果缺乏科学、有效的机制，单纯的基于个例轨迹来实现自动更新，同样很容易让 Skill “过拟合”，陷入局部情况，甚至“越优化越差”。

因此，在企业级的线上任务中应用Agent Skill 时，我们通常不太敢让其直接实现真正意义上进行Online 的自进化。更稳妥的做法是采取Offline 的优化策略，比如：

1.离线收集轨迹数据：在离线环境中收集Agent 执行轨迹，手动进行 Skill 的更新、进化和维护模拟

2.人工审核与评测验证：优化后的Skill 必须经过人工审核和确认，甚至很多场景会建立一套完善的回归评测体系，去验证新版本的 Skill 是否真的带来了效果提升，并且没有引入新的额外问题

3.灰度切流上线：只有在离线验证Skill 确认无误后，才敢去替换线上的版本，并且还要灰度切流，持续观察效果。

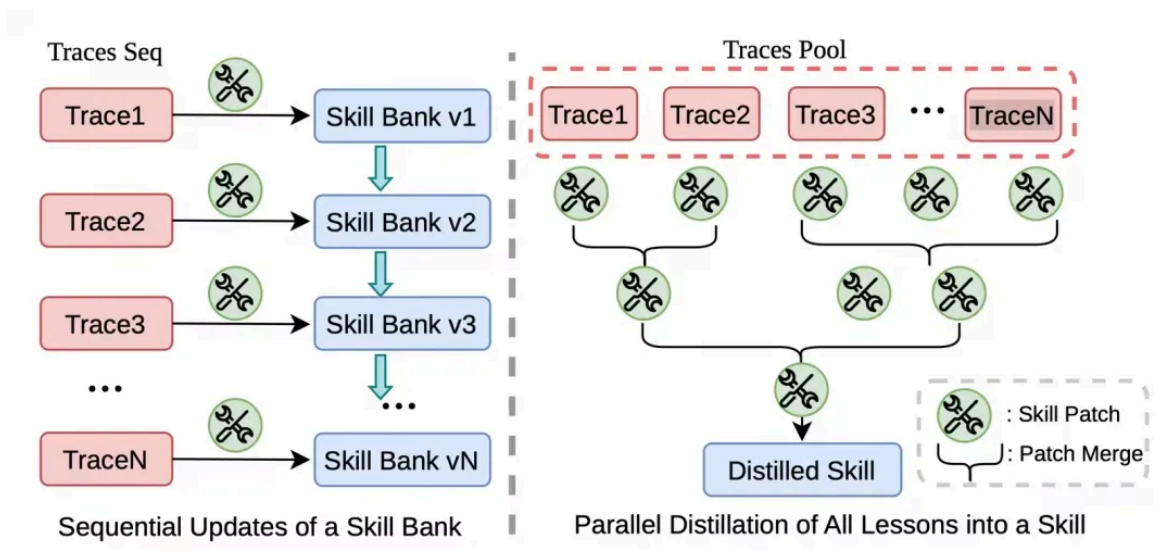
这种“离线优化+ 在线验证”的流程，尽可能去保障了企业级服务的稳定性和可靠性，但如果有大量的这些Skill 需要优化和验证，也都每个像这样人工深度参与，就严重制约生产力，很难“规模化”。并且整个过程从收集轨迹数据、离线优化、评测验证，整个链路非常耗时耗力。因此，严格来说，这种“离线优化”并不是真正意义上的“自进化”，而是由人指导、在离线环境中进行的，Agent 本身并没有具备自主判断和修正的能力，核心的决策权依然在人的手中。

这就引出了本文想要深入探讨的核心话题：**如何更科学、更可控、更保质保量的实现Skill 自进化？**我们追求的不再是那种“无脑”的、方向不稳定的盲目自进化，而是一种能够在保证质量前提下，真正释放人力、实现闭环优化的智能机制。近期，我深入调研了 Skill 自进化领域的相关的论文。其中，有三篇我个人认为比较有里程碑意义的论文，它们代表了三种Skill 的优化思路，并且这三篇论文对应的代码均已开源。对于正在探索Agent Skill 落地的和应用的同学们来说，这无疑是非常好的参考素材。所以，本文我会对这三篇论文进行深度解析，有兴趣的同学，也可以深入去阅读以下论文原文以及 GitHub 上的开源代码，看看哪种方案更契合自己的业务场景。

Trace2Skill：“归纳法”的聚合式进化

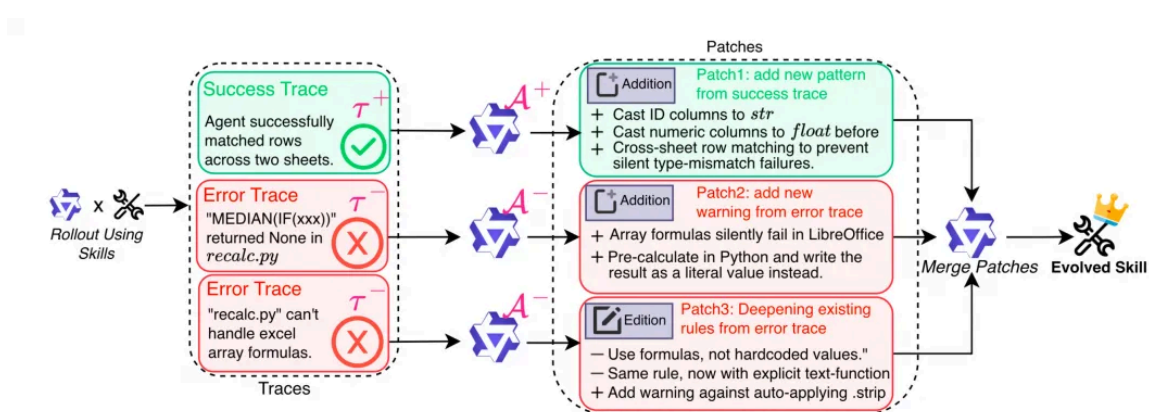
第一个要介绍的论文是来自阿里千问团队的《Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills》，这个项目的名字也叫 Trace2Skill。从名字就能看出来，它的目标就是将**轨迹（Trace）转换为Skill**，同时，它也直击我们前面提到的痛点：**单点Agent 轨迹的局限性**。

就像我在上文中所说的，如果我们仅仅依据某一轮Agent 的执行轨迹来更新 Skill，极易陷入“盲人摸象”的困境，导致 Skill 的方向“跑偏”。因此，Trace2Skill 的核心思路在于**强化“归纳能力”**。它让一支“分析师小分队”**并行地看大量的Agent 执行轨迹**，把每条轨迹里学到的零碎经验合并成一份完整、无冲突的Skill 文档，它模仿了人的工作方式：先广泛积累领域知识，再一次性写出一份成熟的操作手册，而不是边干边改，整理出一堆碎片。



这个图里面，左侧是传统的Online 的 Skill 自进化，传入的单个轨迹的经验会按照顺序依次去更新 Skill 库里的 Skill。而右侧就是 Trace2Skill 的做法，会并行去分析大量轨迹，并以分层方式整合经验教训，从而归纳出可泛化的、标准的 Skill。

Trace2Skill 的核心实现过程可以拆解为三个关键步骤，如下图所示，我们就每个步骤具体来看下。



轨迹生成

这一步的目标是构建高质量的“数据原材料”。

●**初始输入**：给定一份初始Skill，比如是S0（可以是人工编写的，也可以是模型生成的初始草稿）

●**轨迹生成**：让Agent 通过 ReAct 的方式在批用户任务上运行，产生大量的执行轨迹。这个过程是完全可以并行的，效率会比较高，比如，在实践中，使用一个 122B 参数的 LLM 生成 200 条包含 50+ 个轮次的 Agent 轨迹，所需时间不到两个小时

●**正负样本分离**：根据执行结果是否正确，将轨迹严格划分为**成功集 T_+** 和**失败集 T_-** 。这种分层为后续的差异化分析奠定了基础

并行提案

这是Trace2Skill 非常创新性的部分。它不再让一个 LLM 处理所有信息，而是为**每一条轨迹独立分配**一个Sub-Agent 的分析师，输出一个针对该轨迹的“补丁提案”（Patch Proposal）。这里采用了一种不对称的角色设计策略：

●Success Analyst (A^+) – 成功经验提取：

■机制：针对成功集 T^+ 进行分析，一次性调用LLM

■逻辑：成功的案例通常比较容易识别和分析。分析师只需清理轨迹噪声，提取出可泛化的成功行为模式，直接写成补丁即可

■特点：高效、低成本，因为成功路径往往具有较高的一致性

●Error Analyst (A^-) – 失败根因挖掘：

■机制：针对失败集 T^- 进行分析，采用ReAct 多轮循环推理

■逻辑：失败的原因千奇百怪，单次LLM 调用极易产生幻觉或误判。因此， A^- 需要能够读取输入输出文件、对比Ground Truth，并通过迭代验证假设来寻找真正的根因写成补丁

■质量门控：如果经过多轮推理仍无法找到明确根因，该条轨迹会被直接丢弃。这确保了进入下一阶段的都是高质量、有明确改进方向的反馈

■隔离性：所有分析师都基于同一份冻结的初始Skill S_0 进行分析，互不干扰。这种设计保留了观察的多样性，避免了早期错误观点的相互污染

无冲突归纳

最后一步是将分散的补丁池 p 合并成一份最终的更新 p^* 。这本质上是一个层次归并（Hierarchical Merge）的过程：

●递归合并：每层最多 B_{merge} 个补丁合成一个，逐层递归向上，直到合并为最终版本。

●单一模型：整个合并过程由同一个LLM 充当“合并器”，负责去重、解决冲突、保留独特见解。

●硬约束（Hard Constraints）：在合并这些补丁集的时候，会通过代码来做一些硬约束：

■引用检查：如果补丁引用了不存在的文件或变量，直接拒绝；

■冲突标记：如果同一文件的同一行被多个补丁多次编辑，标记为冲突并暂缓处理；

■格式校验：最终生成的Skill 必须通过严格的格式校验器检查，确保语法正确。

在这种层次化的归并中，如果合并器发现那些在多个补丁中反复出现的模式，这就恰恰说明这是系统性问题或通用规律，就会被保留并升级为通用原则；而那些只出现一两次的个例的修改，就很可能只是个例或噪声，则会被果断丢弃。这样，Trace2Skill 就实现了从“单点试错”到“群体归纳”的跨越，有效地解决了Skill 进化中的被个例引导走偏从而失去泛化性问题

举个例子，在我们的阿里云服务领域中，如果我们打算总结某一类问题的通用解决方案，比如“ECS无法远程连接”，仅看1~2个工单，是非常难抽象出完整的、具有普遍性的诊断思路的，因为单个案例往往带有强烈的偶然性和特异性。但如果我们将这一类工单放在一起看，情况就完全不同了。我们可以对大量相似轨迹进行分析、聚合，从中抽取那些符合“最大公约数”的核心内容。比如“CPU/内存跑高”、“磁盘写满”、“端口不通”、“安全组拦截”等等这些检测项才是被多次验证、高频出现的内容，才是真正稳定、有价值的知识沉淀。

结论与思考

为了验证Trace2Skill 的有效性，论文将实验放在极具挑战性的表格领域，用了 **SpreadsheetBench-Verified** 和 **WikiTableQuestions** 来做为Benchmark，重点考察其泛化能力与跨模型迁移性。具体的实验数据在论文中都有，大家可以去看原文，因为篇幅的关系，我这里写一下核心结论，希望大家在构建自主Agent 时提供新的视角。

1. 轻量级的高度可迁移方案

Trace2Skill 可以把复杂的 Agent 经验打包成一份高度可迁移的 Skill，不需要更新模型参数，也不需要引入复杂的外部 RAG 检索模块，只需要使用开源级的模型就可以完成从 Agent 轨迹中构建 Skill。

2. 轨迹分析做出来的 Skill 真的能泛化

Trace2Skill 的实验结果有力地挑战了一个传统假设：“经验本质上是任务特定的，必须通过情景记忆（Episodic Memory）库检索”的假设。实践证明，通过轨迹分析提炼出的Skill 具有泛化能力，是可以跨任务领域迁移的。这说明，一个高质量的Skill 里的逻辑规则比零散的记忆（Memory）更具泛化性。

3. 设计哲学：像人类专家一样“先观察，后总结”

在方法论层面，这种模仿人类专家学习路径的设计哲学还是挺有用的：“先看够多 → 再写一份完整文档”。让Agent 先充分阅读和分析大量轨迹，经过深度的归纳推理后，输出一份结构完整、逻辑自洽的Skill 文档。这种“批处理式”的知识沉淀，往往比“流式”的反应更能捕捉到任务的本质规律。

4. 一些局限性与展望

当然，Trace2Skill 目前的探索也不完善，仍存在局限性，后续可以重点突破的方向：

- 因果贡献难以定量：目前还无法精确量化Skill 中某个具体补丁（Patch）对最终效果提升的贡献值

- 使用率追踪缺失**：对于生成的Skill 文档，还缺乏细粒度的监控手段来追踪各段落在实际运行中的真实调用率和有效性

- 缺乏验证机制**：Skill 的构造过程已经比较像人类专家了，但验证还是需要人来做，还不能自动验证，没有自动化的验证机制，就很难保证自动迭代出来的Skill 是否真的会提升

EvoSkill：“自验证”的自然选择

如果说Trace2Skill 是通过“归纳法”将经验聚合成静态的 Skill，那么 **EvoSkill** 则引入了动态的“验证”过程，虽然从 EvoSkill 名字上似乎只能看出它的核心目标是实现 **Skill 的自进化**。EvoSkill 是由 Sentient Labs 发布的一款开源 Skill 自进化项目，论文标题叫做《EvoSkill: Automated Skill Discovery for Multi-Agent Systems》。

从“构建 → 验证”的架构设计

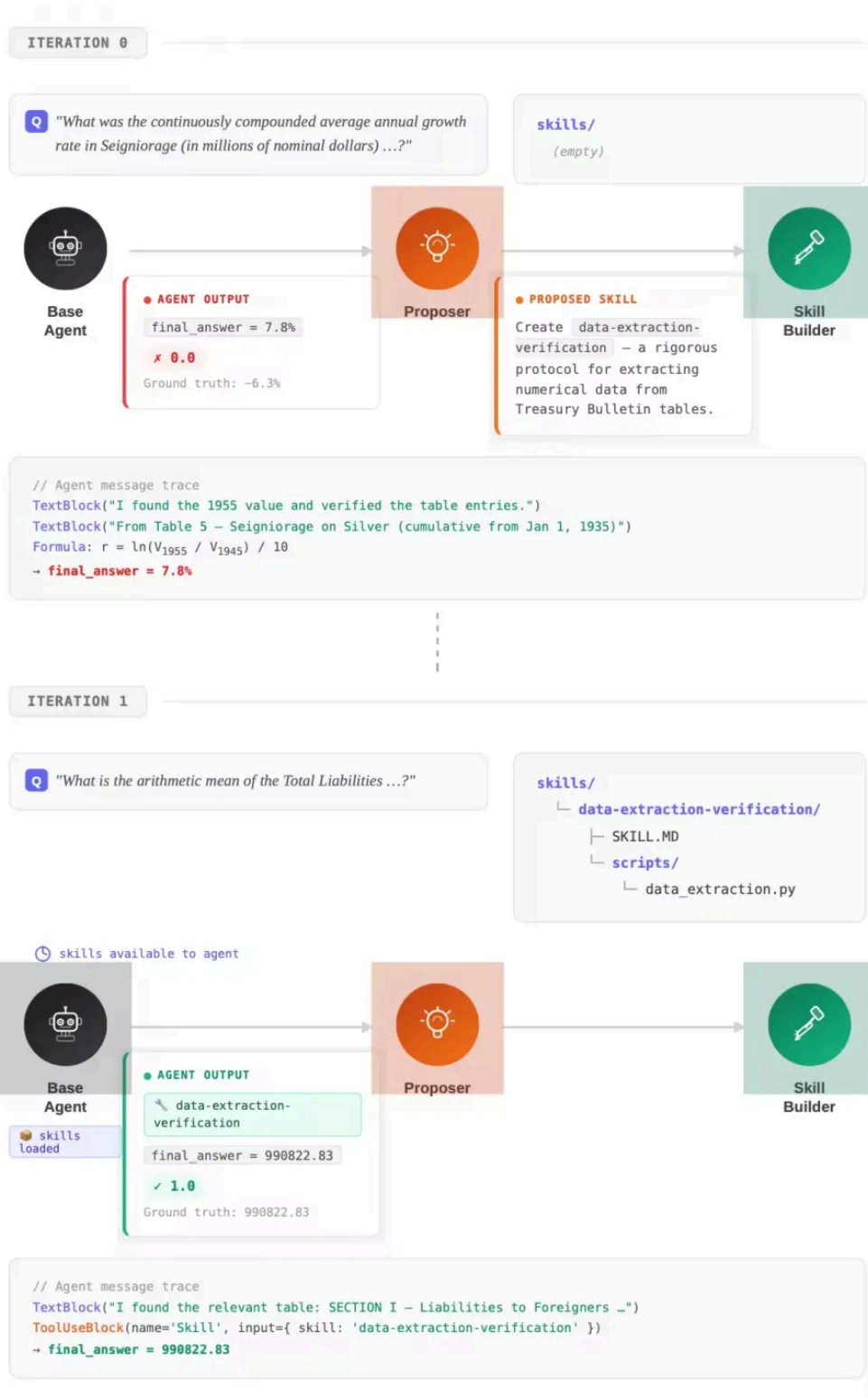
这种从“构建 → 验证”机制的设计灵感，就有点像武侠小说中的“左右互搏”了，也高度抽象了人类专家在离线优化Skill 时的思维过程：不再依赖人工去手动构建和调试，而是构建了一个由三个专用 Agent 组成的闭环系统，让它们相互协作、互相验证，自动完成 Skill 的分析和进化。

EvoSkill 的核心在于将自进化的LLM拆解为三个角色明确的**Sub-Agent**了，形成一个Pipeline：

- 执行者 (Executor)**：负责“跑”。它基于当前待优化的 Skill，去执行具体任务，并产生完整的 Agent 运行轨迹和最终答案。这是进化的“素材来源”，这一步和 Trace2Skill 的第一步是一样的

- 提案者 (Proposer)**：负责“诊”。它深入分析执行者产生的轨迹，判断结果是否正确。如果失败，它需要精准定位问题根源，并提出具体的优化方向和改进提案；如果成功，它也会总结有效模式。这个和 Trace2Skill 里的分析师差有点像，但这里没有去做并行的轨迹分析，并且，EvoSkill 的分析师还会决定是**新建一个Skill** 还是**改一个已有的Skill**，做一个决策

- 搭建者 (Builder)**：负责“改”。它接收分析师提出的优化建议，真正动手编写或修改 Skill 文档，将其落实为可执行的代码或指令规则。



在这个架构之上，EvoSkill 建立了一套严格的验证机制，来确保每一次进化都是正向的，这个是 EvoSkill 最核心的创新点：

- 生成与执行**：搭建者完成Skill 修改后，系统会在一个独立的验证集（Validation Set）上重新运行该 Skill。
- 效果比对**：系统会自动比对优化前后的性能指标。只有当新Skill 在验证集上的表现确实优于旧版本时这次进化才会被确认保留。

●**负向反馈记录**：如果验证结果显示效果没有提升甚至下降，EvoSkill 并不会直接丢弃这次尝试，而是将这一“失败案例”记录下来。这些记录会成为分析师后续学习的宝贵数据，帮助它更准确地识别哪些优化方向是无效的，从而避免在未来的迭代中重蹈覆辙。

进化过程的算法实现

为了保证进化的方向始终朝向更优解，EvoSkill 采用了一种基于“**前沿集合 (Frontier)**”的迭代算法。什么叫**前沿集合**呢？定义为 G ，指的是一个容量固定为 k 的“精英池”，始终保留当前迭代中得分最高的 k 个**程序 (Program)**。等等，你可能会问，这里的**程序**又是什么概念呢？这个程序指的是Agent 能力的完整载体，包括封装了的**系统提示词 (System Prompt)** 和累积的各种**Skills 库**。那么，EvoSkill 的进化的过程，本质上就是用更强的新程序替换池中较弱的旧程序的过程。这个程序非常像 RL 里的**策略 (Policy)**，如果大家了解RL 的话，可能会更好理解一些。

整体进化过程是一个Loop，每一轮迭代 t 都严格遵循以下步骤，来确保进化方向的正确性：

- 1. 选择父代**：从前沿集合 G 中**轮询**选择一个父程序 p ，确保每个成员都有被优化的机会。
- 2. 挖掘失败**：在训练集上运行 p ，收集得分低于阈值的**失败样本集 F** 。若没有失败的样本，则说明当前程序的序已足够强大，可以直接跳过本轮迭代。
- 3. 诊断提案 (Proposer)**：Proposer Agent 结合失败集 F 和历史记录，分析执行轨迹与能力差距，输出文本形式的**优化提案 π** 。
- 4. 落地构建 (Skill-BUILDER)**：Skill-BUILDER Agent 将提案 π 具体化为**候选程序 $p_{\sim}$** （即在父程序基础上新增或修订技能）。
- 5. 严格验证**：在独立的**预留验证集 VV** 上评估 $p_{\sim}$ 。若其得分高于 G 中最弱的成员，则 $p_{\sim}$ 进入集合并取代最弱者；否则丢弃。
- 6. 历史沉淀**：无论成败，都将提案、得分及判决结果记入历史库 h ，供后续迭代参考，避免重蹈覆辙。

最终，经过 T 次迭代，系统输出前沿集合 G 中得分最高的程序，这里面的System Prompt 和 Skill 都是**最优解的版本**。

这样做的好处是可以做**正向筛选**，使用固定容量的前沿集合机制，确保了只有真正带来性能提升的Skill 才能被保留，防止无效优化导致的退化。而针对**失败的尝试并非无用**，它们作为“反面教材”存入历史库 h ，帮助Proposer Agent 学会识别无效的优化路径，从而越进化越聪明。这个过程，基本上就是将人工调试 Skill 的过程给 Agent 化了，为构建具备长期自主进化能力的智能体系统提供了极具参考价值的新范式。

通过这种“**执行 → 提案 → 构建 → 验证**”的闭环，EvoSkill 就实现了 Skill 的自动化迭代。它不仅仅是在更新成功经验或者修补了一些失败错误，更是在不断的这种验证和选择过程中，让 Agent 逐渐摸索出更通用、更优的解题策略。确保了 Agent 在一个能正向提升的方向上“进化”，而不是盲目进化。

结论与思考

EvoSkill 的核心算法其实不复杂，一个里程碑式的创新就是加入了“验证”，那么，这就需要我们思考一下，为什么引入严格的验证机制能让Agent 的效果更可控？本质上，验证就是Agent 进化过程中的“奖励函数”（Reward Function）。这就好比强化学习（RL）中的逻辑：我们需要一个明确的信号来告诉模型“什么是好的”。在EvoSkill 中，验证集上的得分就是这个信号。它也像梯度下降中的损失函数一样，指引着Skill 优化的方向。如果没有这个可量化的“导航仪”，Agent 的迭代就会陷入盲目。

1. 从“体感”到“量化”：打破不可持续的瓶颈

很多Agent 开发者在初期构建 Agent 时，依赖的是“人的体感”——我觉得这个 Prompt 改得更好了，我觉得回答更流畅了。在个人场景下靠体感没问题，因为你就是唯一的用户和裁判。但在企业级场景下，靠体感是不可持续且无法规模化（Scaling）的。当任务复杂度上升，人的主观判断会出现偏差，甚至产生“我觉得写得很清楚，但模型理解有误”的认知错位。

如果没有一套客观、可量化的验证机制（如自动化测试用例、代码执行通过率等），我们就无法区分是“Skill 本身的质量下降”还是“线上环境的其他因素导致了指标波动”，这种模糊性会导致迭代变得盲目且不稳定。只有通过严格的验证，我们才能自信地说：“我的 Skill 质量确实提升了”，或者精准定位问题所在。

2. 可验证性决定迭代速度：数据飞轮的关键

不知道大家有没有发现，尤其是进入2026年后的这几个月，Claude Ops、GPT、Qwen 等头部 LLM 迭代和发布速度越来越快，之所以这些 LLM 能以月甚至以周为单位快速迭代，其中一个比较核心原因就在模型效果的衡量，越来越可验证。例如AI Coding 场景，代码能否跑通、单元测试是否通过，这些都是验证成本较低且结果比较客观的场景。如果每次迭代都需要大量人工标注或主观评估，则迭代周期久会被无限拉长。其实，Agent 也是同理，构建一套高质量、自动化的验证评估体系，才是Agent 效果能“起飞”的前提。一旦验证闭环打通，数据飞轮才能真正转动起来，从“Agent 产生轨迹”→“自动化验证给出即时反馈”→“根据反馈快速调整 Skill”→“新 Skill 再次进入验证循环”。只有在这样的模式下，迭代速度才能从“人力驱动”转变为“算力驱动”，实现质的飞跃。反之，若缺乏可验证的标尺，任何优化都可能是原地踏步，甚至南辕北辙，不进反退。

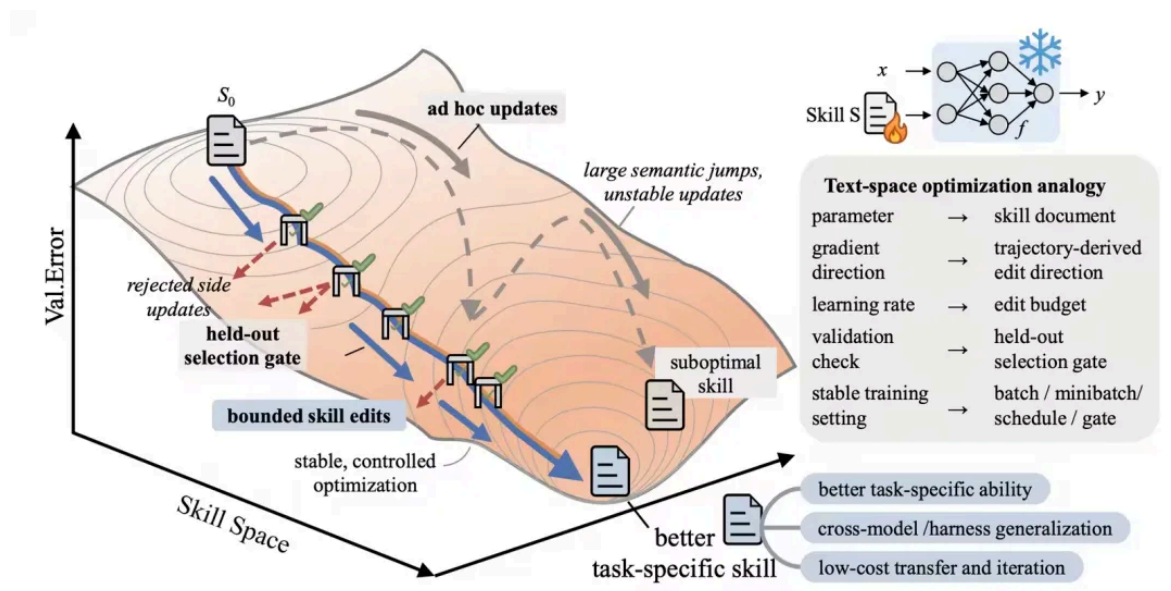
SkillOpt：将 Skill 进化对标为“模型训练”

如果说EvoSkill 是通过“自验证”式和优胜劣汰来进化，那么 SkillOpt 则走了一条更硬核的路线：它将 Skill 的优化过程，直接对标为LLM的参数训练过程。SkillOpt 是由微软公司联合上海交大、同济、复旦等多所高校一起提出的 Skill 优化新范式，论文标题叫做《SkillOpt: Executive Strategy for Self-Evolving Agent Skills》，最近讨论的热度也比较高，我也认为这是一个新的里程碑式的项目。以后，这类基于文本的优化过程，应该也会像模型权重的训练一样，越来越标准化和科学化。

Skill 是“外部可训练参数”

在传统的深度学习范式里，模型主要是通过**梯度下降（Gradient Descent）**不断更新模型的权重（Weights），以最小化损失函数。而SkillOpt 则创造性地提出了一个类比，把“Skill 文本”类比为“模型权重”，把“基于反馈的文本重构”类比为“梯度更新”、把“LLM 驱动的改变引擎”类比为“优化器（Optimizer）”。

在这种视角下，Skill 不再仅仅是一段静态的提示词，而被视为一种“外部的、可训练的文本参数”。SkillOpt 引入了一个专门的“优化器 Agent”，它像 SGD 或 Adam 优化器一样，根据验证集上的表现（即“Loss”），不断地对 Skill 文本进行微调、重写和迭代，直到收敛到一个高质量的状态。



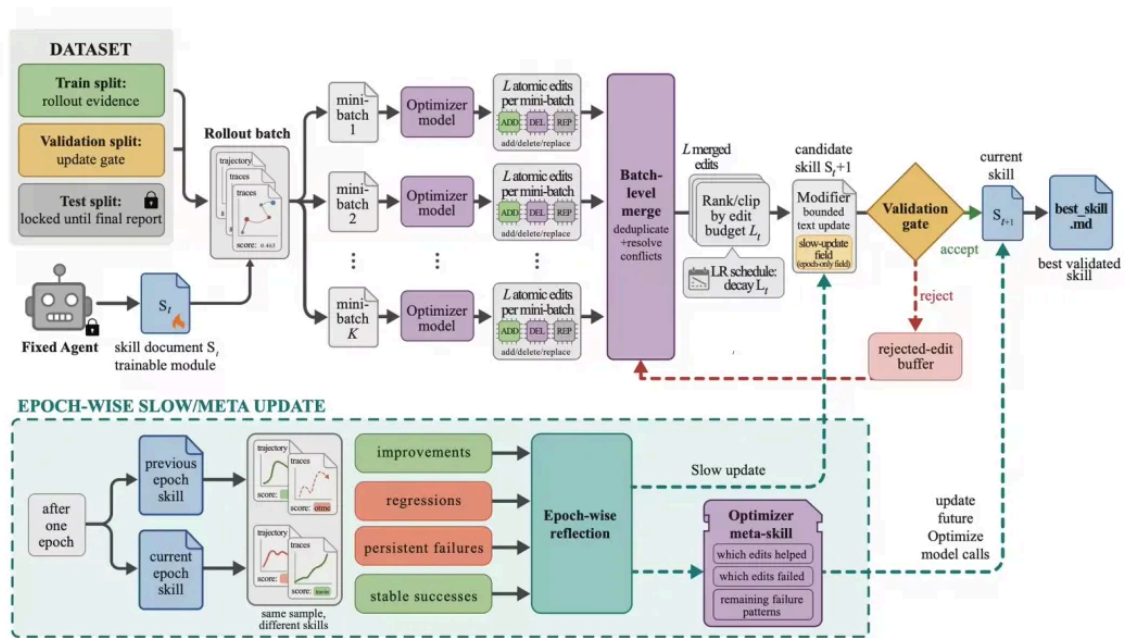
上图是SkillOpt 的一个概览图。目标模型利用当前Skill 执行任务，一个额外的前沿优化器模型将Agent 轨迹转化为有界的添加、删除、替换 Skill 的编辑操作；一个 hold-out 的门控机制仅接受那些能提升验证集效果的修改。被接受的修改会被导出为可复用的Skill，而被拒绝的修改则作为负面反馈用于后续的更新。

那SkillOpt 为什么这样做呢？传统的 Prompt 优化往往依赖于人工调试，缺乏明确的方向感。而 SkillOpt 通过引入“训练范式”，带来了两个关键优势：

- 方向明确**：每一次Skill 的修改，都是基于明确的“误差信号”（即验证失败的原因）。这就像梯度指向了损失函数下降最快的方向，SkillOpt 让文本优化也有了明确的“梯度”，这个逻辑和 EvoSkill 的“验证”思路其实是一样的。
- 系统化迭代**：它摒弃了零敲碎打的修补，而是将整个Skill 作为一个整体对象进行系统性优化。这种“端到端”的训练思维，使得 Skill 的结构更加紧凑、逻辑更加自洽。同时还引入了文本领域的“学习率”（Learning Rate），能够控制每次Skill 优化的粒度，也就是文本修改的比重。

六大核心组件详解

为了将“Skill 即参数”的理念落地，SkillOpt 设计了六大核心组件和 workflow。这套流程严格对标深度学习中的训练范式，从数据收集、梯度计算、参数更新到验证 gating，每一步都经过精心设计，以确保 Skill 优化的稳定性和高效性。



SkillOpt 整体的流程如下图所示，冻结参数的目标模型利用当前 Skill 执行一批 Rollout；优化器模型对成功和失败案例进行小批量的反思，提出有界的添加、删除、替换优化操作，在预定的编辑额度下对这些操作进行合并和排序，并且仅通过预留的 hold-out 验证门控机制来接受候选 Skill。在各个 epoch 中，慢速、元更新在不改变目标模型的情况下保留更长视野的经验教训。具体的细节，我们展开来看下。

1. 前向传播：证据收集 (Forward Pass: Rollout Evidence)

就像模型训练需要先进行前向传播以获取 Loss，SkillOpt 的第一步是让目标模型在训练集上执行任务。这个其实和 Trace2Skill、EvoSkill 的第一步都是一样的，先生成 Agent 轨迹：

- **批量执行：**默认以 Batch Size=40 运行一个 rollout batch。
- **全量记录：**不仅记录最终答案，还完整捕获任务上下文、消息历史、工具调用、观察结果、验证器反馈以及 Harness 特定的元数据（如表格预览、文档引用、执行 Trace 摘要等）。
- **解耦设计：**支持累积多个 Batch 后再统一进行反思，实现了“执行节奏”与“更新节奏”的解耦，提升了工程灵活性。

2. 反向传播：小批量反思 (Backward Pass: Minibatch Reflection)

这就是 SkillOpt 的“梯度计算”环节。优化器模型 (Optimizer) 接收前向传播产生的轨迹，并进行结构化分析。这个和 Trace2Skill、EvoSkill 的分析、提案的步骤基本上也是一样的，但更加细节：

- **分组与分片：**将轨迹分为“成功”和“失败”两组，并进一步切分为 Minibatch（默认大小 8）。
- **为什么用 Minibatch：**单个轨迹容易导致过拟合，产生过于具体的修补 (Over-specific Fix)；而 Minibatch 能暴露反复出现的程序性错误 (Procedural Errors)，从而提炼出更具通用性的规则，这个

有点类似Trace2Skill 多个Batch聚合过程。

- 原子化编辑**：每个Minibatch 产出一组结构化的原子编辑操作，比如追加、添加、删除、替换等。

- 层次化合并**：先分别合并Failure 和 Success 组的编辑建议，最后再做全局合并，且遵循“Failure 优先于 Success”的原则，确保优先解决致命问题。

3. 学习率约束：有界文本更新 (Bounded Text Updates: Learning Rate Constraint)

这是SkillOpt 与传统“无约束 Prompt 重写”最大的区别，也是其稳定性的核心来源。

- 核心创新**：每一步迭代只允许 Lt 条编辑生效。这相当于深度学习中的学习率 (Learning Rate) 控制，控制的越细节，迭代越慢，也越容易陷入局部最优解；但是控制太粗，又可能跳出全局最优解。

- 避免灾难性遗忘**：无约束的重写容易清掉原有有用规则、引入逻辑冲突或过拟合到局部失败案例。通过限制编辑数量，SkillOpt 确保了更新的平滑性。

- 调度策略**：支持Cosine (默认)、Constant、Linear和 Autonomous 四种调度方式。默认的 Cosine 策略从大步长开始，逐渐收敛，模拟了模型训练后期的精细调优过程。

- 编辑模式**：默认采用Patch 模式 (局部微调)，也可切换为 Rewrite 模式 (整体重写)。

4. 验证门控 + 负反馈缓冲 (Validation Gate + Rejected-Edit Buffer)

这一步对应模型训练中的“验证集评估”，但引入了更严格的准入机制：

- 严格优于才接受**：候选Skill 必须在独立的验证集 D_{set} 上运行，只有当得分严格高于当前最优Skill 时才会被接受 (平局也被拒绝)，防止性能抖动。

- 负反馈价值化**：被拒绝的编辑并不会被丢弃，而是连同其导致的分数跌幅一起存入Rejected-Edit Buffer。这些“失败教训”会被喂给后续的优化器，帮助它理解哪些修改方向是危险的，从而避免重蹈覆辙。这将流程从“无条件自我编辑”升级为严谨的 Propose-and-Test 优化闭环。

5. 慢更新 + 元更新：动量机制 (Epoch-Wise Slow/Meta Update)

为了捕捉长期趋势并维持知识稳定性，SkillOpt 引入了类似训练里的动量 (Momentum) 的机制。

- 四类样本归因**：每个Epoch 结束时，用同一批训练样本分别在“上一 Epoch Skill”和“当前 Skill”下重跑，将结果归为四类：Improvements (提升)、Regressions (退步)、Persistent Failures (持续失败)、Stable Successes (稳定成功)。

- 受保护区域更新**：优化器会将纵向的指导原则写入Skill 中的受保护区域，常规的步级编辑无法修改这部分内容，确保了核心逻辑的稳定性。

- Meta-Skill**：维护一份仅对优化器可见、不随Skill 部署的“元记忆”，记录“哪些编辑模式好用”、“哪些被频繁拒绝”、“哪些失败一直未解决”，从而指导优化器自身的策略调整，你可以理解为这是“**调优 Skill 经验的 Skill 的经验**”，套娃了。

6. Harness 无关部署 (Harness-Agnostic Deployment)

最后，SkillOpt 强调了工程落地的简洁性与通用性，是 Harness 无关的：

- 轻量适配**：通过轻量级适配器接口，同一套优化器逻辑可以无缝运行于直接对话、Codex CLI、Claude Code CLI 等多种执行 Harness 框架。

- 极简产物**：最终的部署产物仅仅是一个best_skill.md 文件（通常300 ~ 2000 Tokens），不包含任何复杂的依赖或外部模块。这种“纯文本、零依赖”的特性，使得 Skill 极易移植、版本管理和大规模分发。

通过这六大组件的协同工作，SkillOpt 成功地将非结构化的文本优化问题，转化为一个可控、可解释、可迭代的类训练过程，为 Agent 能力的自动化提升提供了一套标准化的解决方案。

结论与思考

从最后的效果上来看，在SkillOpt 论文中所对比的这些benchmark上来看，SkillOpt 的效果是更好一些的。当然，在实际场景上，还是需要各位同学用的时候进一步做测试和验证。下面，主要总结一下SkillOpt所带来的创新：

- Skill 应当被像参数一样训练**——SkillOpt 是首个系统性、可控的文本空间优化器。

- 关键设计**：边界化的学习率、验证gate、rejected-edit buffer等都被消融实验证明是必需的。

- 产物是真正可部署的**：best_skill.md 是个三百到两千token 的小文档，可以跨模型、跨harness、跨任务迁移。

- 不足的地方**：只自进化调优了best_skill.md 这个单Skill文档，对于其他的文件，比如References 里的Markdown文件，或者 Resources 中的脚本代码等文件没有考虑到自进化中。并且没有考虑跨领域的 Skill 库等等。

三者的对比

最后，在介绍完这三个Skill 自进化的论文之后，我们来做一个总体的对比，让大家更加清晰明了的看到他们的区别。

Trace2Skill：归纳推理学派

- 核心假设**：好的Skill 应该来自对大量轨迹的归纳，单条轨迹的教训不可靠

- 关键动作**：并行处理+ 层次化合并

- 类比**：就像专家开会，大家分头看不同案例，然后开会合并意见，“被多人提到的”才进最终报告

- 优势**：一次成型，效率高，最终Skill 简洁可读

- 风险**：合并器要够强，否则会丢细节

EvoSkill：自验证选择学派

- 核心假设**：Skill 应该慢慢长出来，每轮针对一个具体失败提出改进，能跑赢前沿集合就保留

- 关键动作**：前沿集合+ 失败驱动提案 + 累计反馈历史

- 类比**：自然选择进化，每一代基因有突变（产生新Skill），适应度（验证分数）高的才能延续下来

- 优势**：自然生长出一个Skill 库，每个Skill 都对应一个具体的失败模式，可解释性更强。

- 风险**：每轮只改一处，收敛慢、需要不少迭代；不同轮次跑的结果差异大。

SkillOpt：训练优化器学派

- 核心假设**：Skill 应该被像神经网络参数一样训练，要有严格的约束来保证稳定收敛。

- 关键动作**：学习率约束+ 验证 gate + 负反馈 buffer + 元学习

- 类比**：带momentum + early stopping 的 SGD，每步只动一点点，被拒的更新留作负样本，每轮 epoch 做长期巩固。

- 优势**：可控性最强；每个组件都有明确的解释，并且与神经网络学习对标

- 风险**：组件太多，并且强依赖一个稳定的验证集和打分函数

更多的对比细节，我通过这个表格列出来吧，写的也不一定全，也是按照我个人的理解来的：

对比项	Trace2Skill	EvoSkill	SkillOpt
优化对象	单份 SKILL.md + Reference md 文档	可多个Skill文档	单份 best_skill.md 文档
数据采集	一次性跑完训练集，全部轨迹送进合并器	每轮跑训练 batch，收集失败样本	每步跑 rollout batch（默认 40），区分成功/失败

更新粒度	并行跑 patch，层次化合并所有 patch	每轮一个新 Skill 或一次编辑	每步多个 bounded 原子编辑（新增/修改/替换）
验证过程	编程式格式校验 + 冲突检测（无显式验证）	验证集得分超过前沿集合最弱者才进入	严格大于 当前最优才被接受（平局也拒绝）
失败信号利用	通过 Multi-turn Agent 找根因	Proposer 读 Trace GroundTruth 找根因	分 minibatch 反思失败， 被拒编辑进 buffer 作负反馈
学习率	✗ 一次成型	✗ 每次一个 Skill	✓ 每次Lt条
动量	✗	✗	✓
元学习	✗	累计反馈历史 H（帮 Proposer 不重复出错）	Meta Skill（只给优化器看，沉淀调优经验）
执行 Harness	ReAct 即可	需要底座 Harness	Chat / Codex / Claude Code 等 Harness 无关
模型分离	同一模型自给自足	同一模型扮演三个角色	优化器、目标模型 分离

总结

最后，再次回顾这三种范式，我们实际上见证了Agent Skill 进化正在从“经验主义”走向“科学工程”路径，Trace2Skill 是归纳法的代表，它通过“由点及面”的思维，将分散的单点轨迹提炼为通用的核心思考，解决了从 0 到 1 的快速启动问题。EvoSkill 是自动化离线Pipeline 的进阶，它引入了“分析-验证-更新”的闭环，让 Skill 具备了基于反馈的自进化能力，解决了静态规则无法适应动态场景的痛点。而SkillOpt 则是训练范式的极致体现，它在验证的基础上，引入了学习率、动量等更科学的约束机制，通过小步迭代和严格门控，实现了效果的可控性与稳定性最大化。

那么，哪种方案最好呢？虽然论文的实验数据给出了部分场景中的测试结果，但实际业务场景决定了最终选择。整体来看，引入验证机制的方案会优于纯归纳方案，因为后者在验证的过程中，会引导Agent 不断走向了进化的正确方向。但同时，随着方法复杂度的提升，计算成本和迭代周期也在显著增加。因此，还是需要根据实际业务情况来决策使用哪种方案：

- 如果你的场景相对简单，追求快速落地，并且规律性比较明显，Trace2Skill 的性价比可能最高。
- 如果你对Skill 的效果有明确要求，且拥有完善的自动化评估体系，那么 EvoSkill、SkillOpt 就更适合。
- 也许，混合策略可能是比较好的解法，比如用Trace2Skill 快速生成基线，用 EvoSkill 持续扩充技能库，再对核心瓶颈模块使用 SkillOpt 进行精细打磨。

我这篇文章也是在抛砖引玉，分享一些我在Agent Skill 自进化领域读到的好论文与我个人的思考。当然，技术浪潮滚滚向前，除了这些优秀的项目和论文，我也在持续跟进和学习业界前沿的开源框架与新思想。未来，我将继续围绕“**如何更科学、更可控地构建 Agent**”这一核心命题，与大家共享更多的探索心得。

我也始终相信，随着Agent 评估验证体系的完善和优化范式的成熟，Agent 的构建方法论将从“玄学调试”真正走向“科学工程”。在这个过程中，希望我们大家继续互相交流、共同提升，让 Agent 技术在各行各业业务中发挥出更大的价值。本文是我个人技术探索的心路历程，一家之言、经验之谈，行文仓促，如有错误还请各位批评指正！

References

[1] Trace2Skill 论文: <https://arxiv.org/pdf/2603.25158>

[2] Trace2Skill Github: <https://github.com/Qwen-Applications/Trace2Skill>

[3] EvoSkill 论文: <https://arxiv.org/pdf/2603.02766>

[4] EvoSkill Github: <https://github.com/sentient-agi/EvoSkill>

[5] SkillOpt 官网: <https://microsoft.github.io/SkillOpt/>

[6] SkillOpt 论文: <https://arxiv.org/pdf/2605.23904>

[7] SkillOpt Github: <https://github.com/microsoft/SkillOpt>